

Circular Singly Linked List Implementation in C

Source Code

```
#include <stdio.h>
#include <stdlib.h>

// Structure definition for a Circular Singly Linked List
node
struct Node {
    int val;                // Data value of the node
    struct Node *next;      // Pointer to the next node
};

// Global head pointer and temporary pointer
struct Node *head = NULL, *temp = NULL;

// Function to insert a node at the end of the circular
list
void insertNodeAtLast(int ele) {
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->val = ele;

    // Case 1: Empty list
    if (head == NULL) {
        newNode->next = newNode;
        head = newNode;
    } else {
        temp = head;
        // Traverse to the last node
        while (temp->next != head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = head;
    }
}
```

```

// Function to insert a node at the beginning of the
circular list
void insertNodeAtBegin(int ele) {
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->val = ele;

    // Case 1: Empty list
    if (head == NULL) {
        newNode->next = newNode;
        head = newNode;
    } else {
        temp = head;
        // Traverse to the last node to maintain
circularity
        while (temp->next != head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = head;
        head = newNode;
    }
}

// Function to insert a node after a specific element
int insertNodeAtSpec(int eleToInsert, int atInsert) {
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->val = eleToInsert;

    // Case 1: Empty list
    if (head == NULL) {
        newNode->next = newNode;
        head = newNode;
        return 1;
    }

    temp = head;
    struct Node *prev = NULL;

    // Insert before head if the target element is at the
head
    if (head->val == atInsert) {

```

```

        while (temp->next != head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = head;
        head = newNode;
        return 1;
    }

    // Traverse to find target node
    prev = head;
    temp = head->next;

    while (temp != head && temp->val != atInsert) {
        prev = temp;
        temp = temp->next;
    }

    // Element not found
    if (temp == head) {
        return 0;
    }

    // Insert node after found node
    newNode->next = temp->next;
    temp->next = newNode;
    return 1;
}

// Function to delete the first node
int deleteFirstNode() {
    if (head == NULL) return 0;

    // Case 1: Only one node
    if (head->next == head) {
        free(head);
        head = NULL;
        return 1;
    }

    // Case 2: More than one node

```

```

    temp = head;
    while (temp->next != head) {
        temp = temp->next;
    }
    temp->next = head->next;
    struct Node *del = head;
    head = head->next;
    free(del);
    return 1;
}

// Function to delete the last node
int deleteLastNode() {
    if (head == NULL) return 0;

    struct Node *prev = NULL;
    temp = head;

    // Case 1: Only one node
    if (head->next == head) {
        free(head);
        head = NULL;
        return 1;
    }

    // Traverse to last node
    while (temp->next != head) {
        prev = temp;
        temp = temp->next;
    }
    prev->next = head;
    free(temp);
    return 1;
}

// Function to delete a specific node by value
int deleteSpecNode(int ele) {
    struct Node *prev, *del;

    if (head == NULL) return 0;

```

```

// Case 1: Only one node and match
if (head->next == head && head->val == ele) {
    free(head);
    head = NULL;
    return 1;
}

// Case 2: Node to delete is the head
if (head->val == ele) {
    temp = head;
    while (temp->next != head) temp = temp->next;
    temp->next = head->next;
    del = head;
    head = head->next;
    free(del);
    return 1;
}

// Traverse to find target node
prev = head;
temp = head->next;

while (temp != head && temp->val != ele) {
    prev = temp;
    temp = temp->next;
}

if (temp == head) return 0;

prev->next = temp->next;
free(temp);
return 1;
}

// Function to count total nodes in the circular list
int countNumber() {
    if (head == NULL) return 0;

    int count = 0;
    temp = head;

```

```

        do {
            count++;
            temp = temp->next;
        } while (temp != head);

        return count;
    }

// Function to traverse the list in forward direction
int traverseCircularList(struct Node *head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return 0;
    }

    temp = head;
    do {
        printf("%d -> ", temp->val);
        temp = temp->next;
    } while (temp != head);
    printf("(back to head)\n");
    return 1;
}

// Function to search an element in the list
int search(int ele) {
    if (head == NULL) return 0;

    temp = head;
    do {
        if (temp->val == ele) return 1;
        temp = temp->next;
    } while (temp != head);

    return 0;
}

// Function to traverse the list in reverse using recursion
void traverseReverse(struct Node *head, struct Node
*current) {
    if (current->next != head)

```

```

        traverseReverse(head, current->next);
    printf("%d -> ", current->val);
}

// Main function implementing a menu-driven interface
int main() {
    int choice1, choice2, ele, atInsert, count;

    do {
        printf("\n1.Insert an Element");
        printf("\n2.Delete an Element");
        printf("\n3.Traverse List");
        printf("\n4.Search Element");
        printf("\n5.Count Elements");
        printf("\n6.Reverse Traverse");
        printf("\n7.Exit");
        printf("\n\nEnter Your Choice: ");
        scanf("%d", &choice1);

        switch (choice1) {
            case 1:
                printf("\n1.Insert at Begin");
                printf("\n2.Insert at Last");
                printf("\n3.Insert at Specific");
                printf("\nEnter Your Choice: ");
                scanf("%d", &choice2);

                printf("\nEnter the Element: ");
                scanf("%d", &ele);

                if (choice2 == 1)
                    insertNodeAtBegin(ele);
                else if (choice2 == 2)
                    insertNodeAtLast(ele);
                else if (choice2 == 3) {
                    printf("\nEnter element after which to
insert: ");

                    scanf("%d", &atInsert);
                    insertNodeAtSpec(ele, atInsert);
                }
                break;

```

```

case 2:
    printf("\n1.Delete at Begin");
    printf("\n2.Delete at Last");
    printf("\n3.Delete Specific");
    printf("\nEnter Your Choice: ");
    scanf("%d", &choice2);

    if (choice2 == 1)
        deleteFirstNode();
    else if (choice2 == 2)
        deleteLastNode();
    else if (choice2 == 3) {
        printf("\nEnter element to delete: ");
        scanf("%d", &ele);
        deleteSpecNode(ele);
    }
    break;

case 3:
    printf("\n\n==== List Contents
====\n\n");
    traverseCircularList(head);
    break;

case 4:
    printf("\nEnter element to search: ");
    scanf("%d", &ele);
    if (search(ele))
        printf("\nElement Found\n");
    else
        printf("\nElement Not Found\n");
    break;

case 5:
    count = countNumber();
    printf("\nNumber of Nodes: %d\n", count);
    break;

case 6:
    if (head == NULL)

```



```
        printf("\nList is Empty\n");
    else {
        printf("\nReverse Traversal:\n");
        traverseReverse(head, head);
        printf("(back to head)\n");
    }
    break;
}
} while (choice1 != 7);

return 0;
}
```

Output :

```
1.Insert an Element
2.Delete an Element
3.Traverse List
4.Search Element
5.Count Elements
6.Reverse Traverse
7.Exit
```

Enter Your Choice: 1

```
1.Insert at Begin
2.Insert at Last
3.Insert at Specific
Enter Your Choice: 1
```

Enter the Element: 10

```
1.Insert an Element
2.Delete an Element
3.Traverse List
4.Search Element
5.Count Elements
6.Reverse Traverse
7.Exit
```

Enter Your Choice: 1

```
1.Insert at Begin
```

```
2.Insert at Last
3.Insert at Specific
Enter Your Choice: 2
```

Enter the Element: 20

```
1.Insert an Element
2.Delete an Element
3.Traverse List
4.Search Element
5.Count Elements
6.Reverse Traverse
7.Exit
```

Enter Your Choice: 1

```
1.Insert at Begin
2.Insert at Last
3.Insert at Specific
Enter Your Choice: 3
```

Enter the Element: 20

Enter element after which to insert: 20

```
1.Insert an Element
2.Delete an Element
3.Traverse List
4.Search Element
```


5.Count Elements
6.Reverse Traverse
7.Exit

Enter Your Choice: 3

==== List Contents ====

10 -> 20 -> 20 -> (back to head)

1.Insert an Element
2.Delete an Element
3.Traverse List
4.Search Element
5.Count Elements
6.Reverse Traverse
7.Exit

Enter Your Choice: 4

Enter element to search: 30

Element Not Found

1.Insert an Element
2.Delete an Element
3.Traverse List
4.Search Element

5.Count Elements
6.Reverse Traverse
7.Exit

Enter Your Choice: 5

Number of Nodes: 3

1.Insert an Element
2.Delete an Element
3.Traverse List
4.Search Element
5.Count Elements
6.Reverse Traverse
7.Exit

Enter Your Choice: 6

Reverse Traversal:

20 -> 20 -> 10 -> (back to head)

1.Insert an Element
2.Delete an Element
3.Traverse List
4.Search Element
5.Count Elements
6.Reverse Traverse
7.Exit

Enter Your Choice: 3

===== List Contents =====

10 -> 20 -> 20 -> (back to head)

- 1.Insert an Element
- 2.Delete an Element
- 3.Traverse List
- 4.Search Element
- 5.Count Elements
- 6.Reverse Traverse
- 7.Exit

Enter Your Choice: 2

- 1.Delete at Begin
 - 2.Delete at Last
 - 3.Delete Specific
- Enter Your Choice: 1

- 1.Insert an Element
- 2.Delete an Element
- 3.Traverse List
- 4.Search Element
- 5.Count Elements
- 6.Reverse Traverse
- 7.Exit

Enter Your Choice: 3

===== List Contents =====

20 -> 20 -> (back to head)

- 1.Insert an Element
- 2.Delete an Element
- 3.Traverse List
- 4.Search Element
- 5.Count Elements
- 6.Reverse Traverse
- 7.Exit

Enter Your Choice: 2

- 1.Delete at Begin
 - 2.Delete at Last
 - 3.Delete Specific
- Enter Your Choice: 3

Enter element to delete: 20

- 1.Insert an Element
- 2.Delete an Element
- 3.Traverse List
- 4.Search Element
- 5.Count Elements

6.Reverse Traverse

7.Exit

Enter Your Choice: 3

===== List Contents =====

20 -> (back to head)

1.Insert an Element

2.Delete an Element

3.Traverse List

4.Search Element

5.Count Elements

6.Reverse Traverse

7.Exit

Enter Your Choice: 4

Enter element to search: 20

Element Found

1.Insert an Element

2.Delete an Element

3.Traverse List

4.Search Element

5.Count Elements